Authors:

- Cezary Suchorski 160310
- Bartosz Tarnachowicz 160304

Code repo: https://github.com/csuchorski/evolutionary_computation

Hamiltonian cycle through a selected half of the nodes

Evolutionary Computation · Laboratory 2

This notebook is the report. The optimisation engine is written in C++ (`src/main.cpp`); it reads the instances, builds the distance matrix once and writes machine-readable results (`results/*.csv`). Everything shown below — tables, distributions, 2-D visualisations, best-tour listings and the discussion — is produced here in Python from those files. The construction methods of Laboratory 1 are kept unchanged as a reference and extended with the regret heuristics of Laboratory 2.

Instances	`TSPA.csv` , `TSPB.csv` — 200 nodes each
Selected nodes	$k = \lceil n/2 \rceil = 100$
Solutions	200 random, and 200 per deterministic method (one start per node)
Seed	12345

1. Problem description

Each instance describes **n = 200** points in the plane. Point i has coordinates (x_i, y_i) and an integer cost c_i. We must select exactly **k = ⌈n/2⌉ = 100** nodes and arrange them in one closed tour that visits every selected node exactly once, minimising

$$F(T) = \Sigma_{i=0}^{k-1} [\, c_{t_i} + d(t_i, t_{i+1}) \,], \quad t_k = t_0,$$

where d(i, j) is the Euclidean distance rounded to the nearest integer.

Both terms matter. A cheap node far from the current route may cost more in travel than it saves, and a geometrically convenient node may be expensive. Because $|T|$ is fixed, the node-cost term is an additive penalty of constant size, so the search is never pushed towards selecting fewer nodes.

2. Data and distance matrix

The files contain one row per node with the fields `x;y;cost` . The distance matrix is computed **once, immediately after reading**:

$$d(i, j) = \text{round}(\sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)})$$

with `std::llround` (half away from zero). The optimisation routines receive only `d[][]` and `cost[]` ; the coordinates are used by this notebook for plotting only.

The matrix is stored as **directed arcs** and no symmetry is assumed anywhere. Both supplied instances are Euclidean, hence d(i, j) = d(j, i), but the algorithms never rely on that property.

3. Algorithms

Each method constructs a sequence `T` of exactly `k` distinct nodes. Candidates are always ranked by their effect on the complete objective, never by distance alone.

Notation. `s` is the starting node, `cost[j]` is the cost of node `j`, `d[a][b]` is the distance from `a` to `b`, and `used[j]` says whether node `j` is already in `T`. The objective is evaluated only after construction:

$F(T) = \Sigma_{v \in T}\, \text{cost}[v] + \Sigma_{i=0}^{|T|-1}\, d(T[i], T[(i+1) \bmod |T|])$.

Ties are resolved deterministically by the smallest node index and then by the earliest insertion position.

3.1 Random solution

Choose `k` distinct nodes and visit them in a uniformly random order; there is no greedy scoring.

```
RANDOM(n, k):
    T ← k distinct nodes chosen uniformly from {0, …, n−1}
    SHUFFLE(T)
    return T
```

For the experiment this constructor is called 200 times.

3.2 Nearest neighbour, appending only at the end

The path has a single growing point: its last node. Each step adds the unused node with the smallest increase in the objective.

```
NN_APPEND(s, k):
    T ← [s]
    mark s as used
    while |T| < k:
        j* ← argmin ( cost[j] + d[T[last]][j] ) over unused j
        append j* to T
        mark j* as used
    return T
```

Only the forward arc `T[last] → j*` is added during construction. The closing arc `T[last] → T[0]` is part of `F(T)` and is added when the tour is evaluated.

3.3 Nearest neighbour, best of all positions

A new node may enter at the front, between two consecutive nodes, or at the end. For each unused node and each position, the exact change in the objective is:

```
INSERTION_DELTA(j, p):
    if p = 0:          return cost[j] + d[j][T[0]]
    if p = |T|:        return cost[j] + d[T[last]][j]
    a ← T[p−1];  b ← T[p]
    return cost[j] + d[a][j] + d[j][b] − d[a][b]
```

The constructor then takes the best `(node, position)` pair:

```
NN_ANY_POSITION(s, k):
    T ← [s]
    mark s as used
    while |T| < k:
        (j*, p*) ← argmin INSERTION_DELTA(j, p)
                    over unused j and p = 0, …, |T|
        insert j* at position p*
        mark j* as used
    return T
```

Inserting at the front adds only `j → T[0]`; appending adds only `T[last] → j`; an interior insertion replaces one arc `a → b` by `a → j` and `j → b`.

3.4 Greedy cycle

The cycle is closed after every insertion. A new node is placed into an existing edge `a → b`, so the old arc is removed and two new arcs are added.

```
CYCLE_DELTA(a, b, j):
    return cost[j] + d[a][j] + d[j][b] − d[a][b]

GREEDY_CYCLE(s, k):
    j* ← argmin ( cost[j] + d[s][j] + d[j][s] ) over j ≠ s
    T ← [s, j*]                          # two-node cycle s → j* → s
    mark s and j* as used
    while |T| < k:
        best ← +∞
        for each unused j:
            for each edge a → b of T:   # includes the wrap edge last → first
                Δ ← CYCLE_DELTA(a, b, j)
                if Δ < best:
                    best ← Δ;  (j*, a*) ← (j, a)
        insert j* immediately after a*
        mark j* as used
    return T
```

The wrap edge is treated like any other edge. Inserting at the end therefore costs

cost[j] + d[last][j] + d[j][first] − d[last][first],

not merely `cost[j] + d[last][j]`: the return arc is added and the previous closing arc is removed.

Because every method simply builds `T`, the same `EVALUATE(T)` routine computes the distance, node costs and objective for all 200 runs of every method.

3.5 Greedy 2-regret heuristic

For every unused node we look at **all** insertion positions and record the two cheapest deltas. The *regret* of a node is the gap between its second-best and its best option: a node whose only good position is

much better than every alternative is urgent, because delaying it would be costly. At each step the node with the **largest regret** is inserted at its best position. Because the node cost `cost[j]` is added identically at every position, it cancels out of the difference, so the regret measures only how the **distance** changes: the pure 2-regret rule is cost-blind and ranks nodes by the spread of their distance options alone.

The same rule runs on two bases:

- **open path** — the extended nearest-neighbour insertion set (front, interior, end);
- **closed cycle** — the greedy-cycle insertion set (every edge, wrap edge included).

```
TWO_BEST_DELTAS(j, T):
    D ← sorted list of INSERTION_DELTA(j, p) over all positions p of T
    return (D[0], D[1])              # best, second best

REGRET(base, s, k):
    T ← seed(s, base)                # [s] for a path, the best 2-cycle for a cycle
    mark the seed nodes as used
    while |T| < k:
        for each unused j:
            (best_j, second_j) ← TWO_BEST_DELTAS(j, T)
            regret_j ← second_j − best_j
        j* ← argmax regret_j
        insert j* at its best position
        mark j* as used
    return T
```

On the open path the first step has a single effective position, so every regret is 0 and the rule falls back to the best change (plain nearest neighbour); from the second step on there are at least two positions.

3.6 Greedy heuristic with a weighted sum criterion

The weighted sum combines both signals: the 2-regret and the best change of the objective. Because the two quantities are measured on different scales, each is first normalised to [0, 1] across the candidate nodes at the current step:

$\text{score}_j = w_r \cdot \text{regret}_j^{\text{norm}} + w_b \cdot (1 - \text{best}_j^{\text{norm}})$.

The node with the largest score is inserted at its best position. With `w_r = 1, w_b = 0` the rule is pure regret; with `w_r = 0, w_b = 1` it reduces exactly to the extended nearest neighbour (open path) or to the greedy cycle (closed cycle). By default the weights are equal, `w_r = w_b = 0.5`.

```
WEIGHTED_SUM(base, s, k, wr, wb):
    T ← seed(s, base)
    mark the seed nodes as used
    while |T| < k:
        for each unused j:
            (best_j, second_j) ← TWO_BEST_DELTAS(j, T)
            regret_j ← second_j − best_j
        normalise best and regret over the candidates to [0, 1]
        for each unused j:
            score_j ← wr · regret_j_norm + wb · (1 − best_j_norm)
        j* ← argmax score_j
        insert j* at its best position
        mark j* as used
    return T
```

Ties are broken by the smallest node index, as before.

```
Instance TSPA: n = 200, k = 100
  Random                       min 235605   max 292306   avg 261429.4    (0.6 ms)
  Nearest neighbour (append at end) min 84362    max 90075    avg 87476.6     (7.5 ms)
  Nearest neighbour (any position) min 72328    max 77022    avg 74347.0     (283.8 ms)
  Greedy cycle                 min 72590    max 75281    avg 73793.4     (515.1 ms)
  2-regret (open path)         min 110950   max 135771   avg 121360.3    (305.6 ms)
  2-regret (greedy cycle)      min 112806   max 128089   avg 120378.8    (636.1 ms)
  Weighted sum (open path)     min 85598    max 100718   avg 92603.3     (310.5 ms)
  Weighted sum (greedy cycle)  min 81173    max 100230   avg 91064.1     (642.1 ms)
Instance TSPB: n = 200, k = 100
  Random                       min 183300   max 229903   avg 207269.3    (0.6 ms)
  Nearest neighbour (append at end) min 56007    max 63166    avg 59397.3     (10.4 ms)
  Nearest neighbour (any position) min 52915    max 66164    avg 59475.5     (255.1 ms)
  Greedy cycle                 min 55655    max 63734    avg 59649.9     (517.6 ms)
  2-regret (open path)         min 68761    max 79756    avg 73649.8     (307.4 ms)
  2-regret (greedy cycle)      min 68150    max 77274    avg 72484.8     (639.1 ms)
  Weighted sum (open path)     min 57299    max 66918    avg 61457.6     (323.8 ms)
  Weighted sum (greedy cycle)  min 59136    max 69940    avg 62870.4     (647.8 ms)

Results written to results/
  summary.csv, runs.csv, best_tours.csv, best_tours_columns.csv

Python 3.12.3 · pandas 3.1.0 · matplotlib 3.11.2
```

	Instance	File	n	k	Min cost	Max cost
0	TSPA	TSPA.csv	200	100	0	1,997
1	TSPB	TSPB.csv	200	100	3	999

4. Computational experiment

For every instance, 200 random solutions were generated and every deterministic method was run 200 times. With n = 200 and 200 runs, run *r* starts from node *r*, so every node is used exactly once as a starting point. The random generator is `std::mt19937` seeded with 12345, which makes the results reproducible. The regret and weighted-sum methods are deterministic given the starting node, so their 200 runs also cover every node exactly once. All runs are independent; no solution is improved after construction. The engine is compiled with `g++ -O2 -std=c++17`.

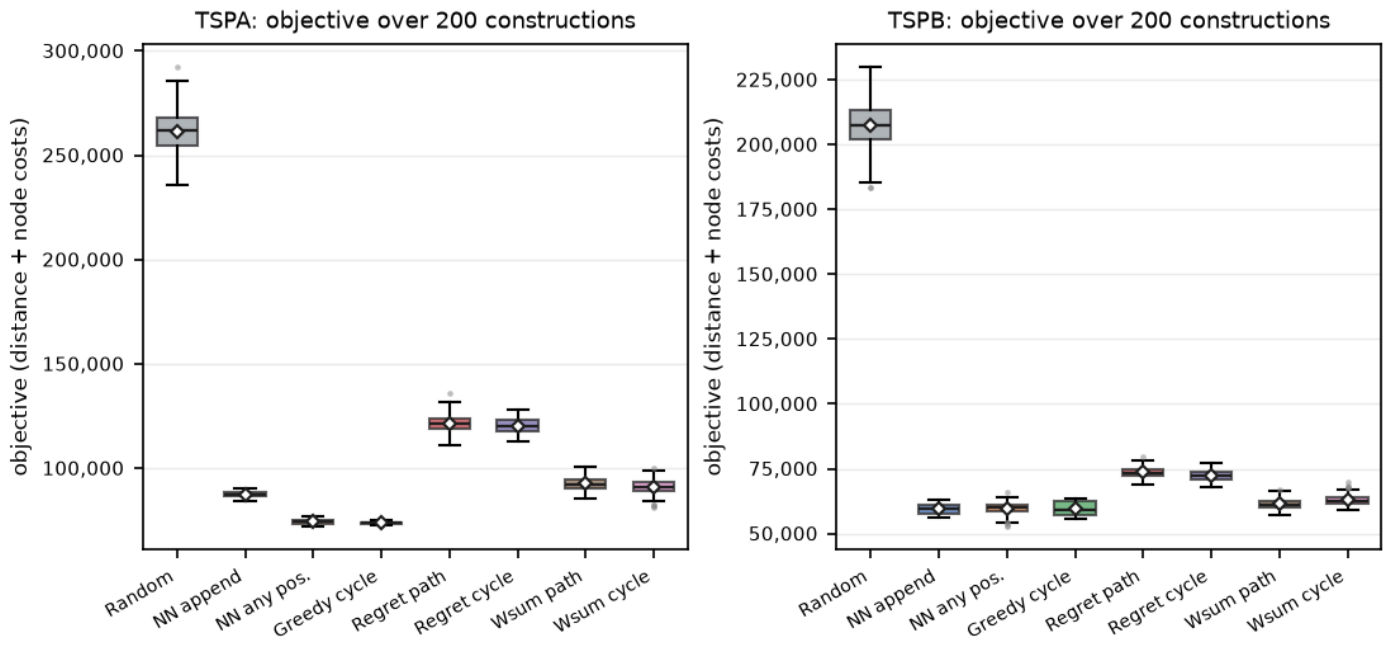
5. Results

The table gives the minimum, maximum and average objective over the 200 constructions, with the two instances side by side.

Instance	TSPA			TSPB		
Statistic	**Min**	**Max**	**Average**	**Min**	**Max**	**Average**
Random	235,605	292,306	261,429.4	183,300	229,903	207,269.3
NN append	84,362	90,075	87,476.6	56,007	63,166	59,397.3
NN any pos.	72,328	77,022	74,347.0	52,915	66,164	59,475.5
Greedy cycle	72,590	75,281	73,793.4	55,655	63,734	59,649.9
Regret path	110,950	135,771	121,360.3	68,761	79,756	73,649.8
Regret cycle	112,806	128,089	120,378.8	68,150	77,274	72,484.8
Wsum path	85,598	100,718	92,603.3	57,299	66,918	61,457.6
Wsum cycle	81,173	100,230	91,064.1	59,136	69,940	62,870.4

NN = nearest neighbour; "append" inserts only at the end of the path, "any pos." at the front, inside or at the end.

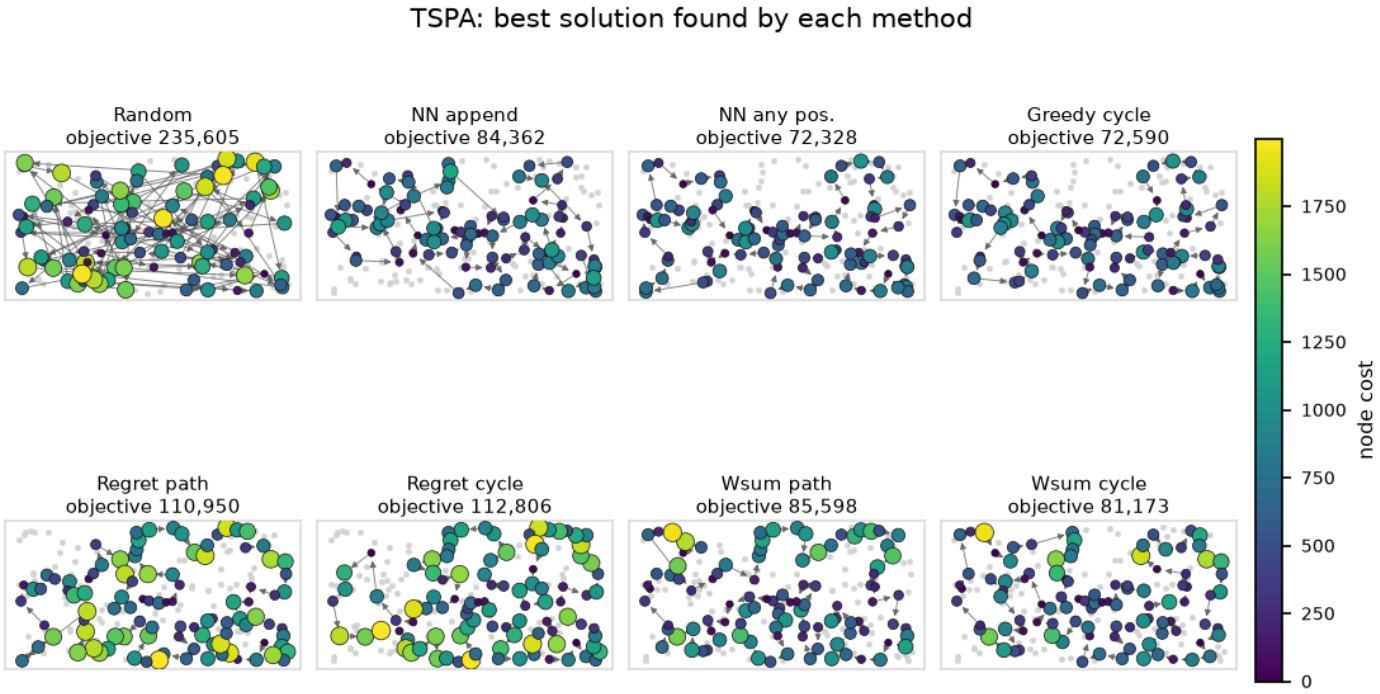
The minimum, maximum and average are the required statistics; the box plots below show the whole distribution of the 200 solutions per method. White diamonds mark the means, horizontal lines the medians.

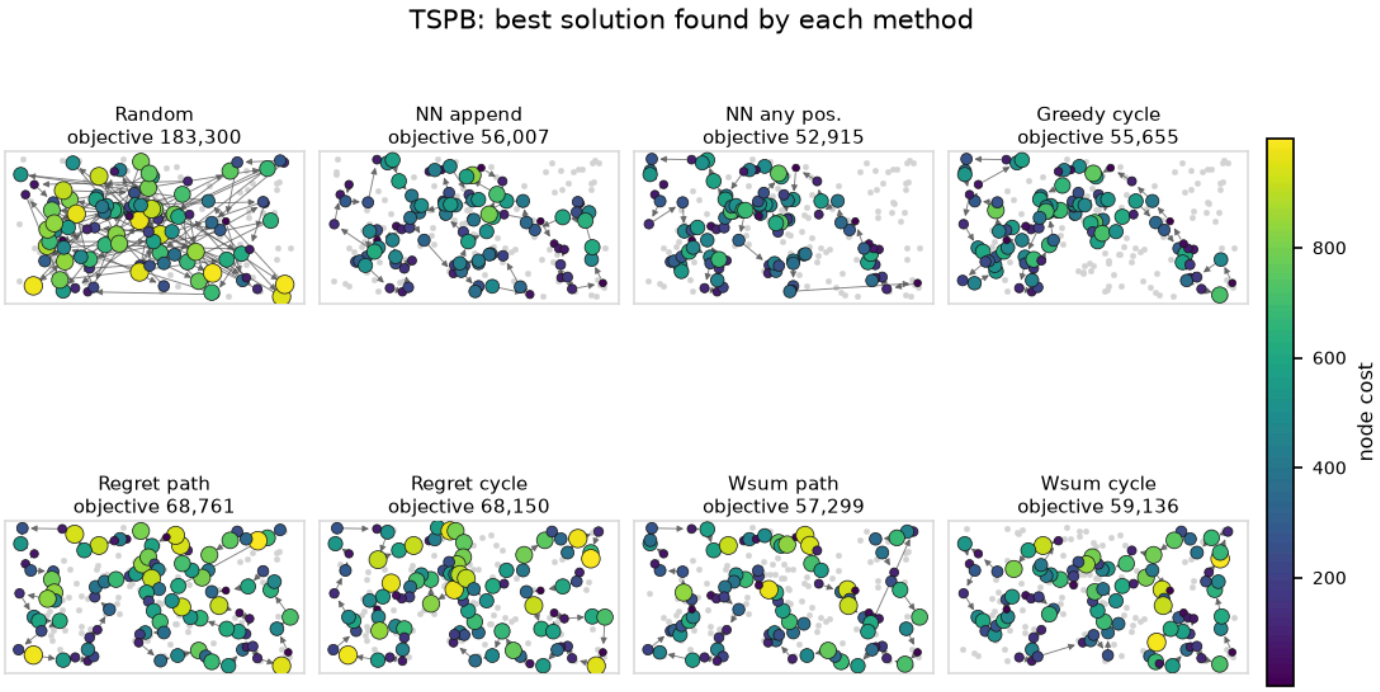


Random construction is far behind the greedy methods and has the widest spread. Among the greedy methods the differences are much smaller, so the choice of insertion rule matters more than the starting node.

6. Best solutions

Each panel shows the cycle built by one method. Grey dots are nodes that were **not** selected. Selected nodes are drawn with a colour **and** a radius that both encode the node cost (dark purple, small = cheap; yellow, large = expensive). Thin arrows show the direction of travel along the cycle.





7. Best solutions as node lists

All indices are **0-based**; the list is the cycle order and the closing arc runs from the last node back to the first one. For spreadsheet use, `results/best_tours_columns.csv` stores the same sixteen tours as columns: every column contains 100 node indices, one per row, so it can be copied directly into Excel without reformatting.

The best solutions below are given in 0-based order and are formatted for the Excel solution checker (`results/best_tours_columns.csv` can be pasted directly). Every stored solution is also independently re-evaluated by the engine's `evaluate()` routine, which recomputes distance, node costs and objective and rejects any mismatch; the external Excel checker had not been run when this report was generated.

TSPA

Random — objective 235,605 (distance 143,298, costs 92,307), first node 40:
  40 113 74 96 71 102 31 137 51 22 92 66 171 32 127 194
  143 15 37 144 193 161 85 41 136 108 29 75 13 155 79 76
  89 45 30 10 149 180 154 185 35 133 129 70 132 16 106 151
  176 156 116 188 57 64 2 111 107 192 158 163 179 36 59 82
  120 124 162 54 183 5 72 55 84 197 196 157 153 186 99 168
  177 18 4 123 119 166 112 141 135 65 187 198 1 109 93 46
  17 95 195 199

NN (append at end) — objective 84,362 (distance 31,319, costs 53,043), first node 54:
  54 9 33 36 177 67 144 53 135 58 134 176 174 106 89 107
  57 182 151 93 50 3 1 164 178 29 136 186 184 47 159 130
  153 78 88 193 170 146 75 64 150 42 113 43 85 142 8 131
  84 158 192 181 114 141 180 132 76 80 48 31 63 149 168 97
  127 187 30 123 152 108 60 116 49 11 119 160 83 90 51 16
  35 87 45 140 52 179 185 28 102 156 157 71 91 38 100 4
  112 139 115 5

NN (any position) — objective 72,328 (distance 21,816, costs 50,512), first node 104:
  104 129 69 127 97 168 116 149 63 11 49 160 83 123 100 152
  108 60 31 48 80 91 157 71 4 76 132 180 28 102 185 35
  156 51 16 67 144 140 53 179 52 177 36 33 9 54 131 174
  8 164 186 136 163 29 101 178 1 169 3 50 147 47 93 106
  42 89 113 130 159 146 170 153 193 78 88 107 64 57 182 151
  150 176 72 134 84 58 158 192 181 114 135 73 45 87 141 187
  30 189 2 171

Greedy cycle — objective 72,590 (distance 21,570, costs 51,020), first node 31:
  31 116 149 168 63 11 49 60 108 152 100 90 123 83 160 97
  127 69 171 2 189 30 187 141 87 45 73 135 114 181 192 158
  58 84 134 72 176 150 151 182 57 64 107 88 78 193 153 170
  146 159 130 113 89 42 106 93 47 147 50 3 169 1 178 101
  29 163 136 186 164 142 8 174 131 54 9 33 36 177 52 179
  53 140 144 67 16 51 156 35 185 102 28 180 132 76 4 71
  157 91 80 48

2-regret (open path) — objective 110,950 (distance 21,279, costs 89,671), first node 104:
  104 69 120 97 168 149 11 49 108 152 100 123 148 162 132 76
  4 157 18 173 16 156 28 185 67 144 179 177 36 33 143 190
  139 112 5 115 175 39 154 34 109 186 136 197 163 19 178 101
  155 21 147 37 47 50 93 81 43 113 117 130 159 170 75 26
  78 122 107 64 57 105 182 111 138 165 151 150 176 72 65 134
  84 58 158 192 181 118 114 25 121 73 45 70 96 10 183 166
  32 40 30 23

2-regret (greedy cycle) — objective 112,806 (distance 21,429, costs 91,377), first node 189:
  189 166 183 10 70 22 25 114 118 181 121 61 41 140 53 179
  177 36 99 84 134 72 65 20 192 111 182 165 151 106 89 124
  57 64 107 88 78 26 75 170 159 130 117 93 50 47 37 147
  6 46 86 55 17 29 19 163 197 136 195 184 164 142 85 81
  174 194 9 33 143 62 34 154 39 175 115 5 112 139 18 4
  196 51 190 67 185 156 28 180 191 141 30 160 80 82 11 168
  188 120 125 187

Weighted sum (open path) — objective 85,598 (distance 22,102, costs 63,496), first node 123:
  123 132 76 4 51 139 112 5 115 175 39 154 62 34 186 136
  24 163 19 29 55 101 46 59 1 3 50 93 47 130 159 170
  75 107 64 57 182 111 138 151 89 106 150 174 176 72 134 158
  192 58 84 99 131 9 33 36 177 52 144 53 140 41 73 135
  181 114 25 98 27 87 102 185 67 35 180 141 30 187 189 2
  171 92 161 120 69 127 97 168 63 149 60 108 152 66 80 91
  137 126 48 31

Weighted sum (greedy cycle) — objective 81,173 (distance 21,974, costs 59,199), first node 54:
  54 8 174 81 93 50 3 1 164 109 186 136 184 178 29 17
  101 155 21 147 37 47 130 159 146 107 88 64 57 182 111 138
  151 89 106 150 176 72 134 84 58 158 192 181 114 135 73 45
  98 27 87 141 28 180 76 132 100 152 123 160 30 187 161 2
  171 92 120 127 97 168 63 149 60 31 48 126 80 38 91 157

115 5 112 139 18 196 51 16 35 185 67 144 53 179 52 177
36 131 33 9

TSPB

Random — objective 183,300 (distance 133,397, costs 49,903), first node 168:
168 15 169 25 195 22 147 189 177 102 161 60 181 59 57 149
158 184 75 138 163 160 192 23 172 62 26 123 120 52 171 134
128 198 137 194 175 19 98 85 114 90 33 105 162 34 101 109
113 108 183 135 8 196 141 143 31 79 117 170 28 91 150 151
47 186 164 45 155 94 190 54 74 130 191 53 179 4 156 29
140 27 2 21 69 20 76 119 39 115 49 30 121 100 104 13
166 167 43 55

NN (append at end) — objective 56,007 (distance 24,632, costs 31,375), first node 32:
32 89 57 76 54 127 129 90 153 20 14 2 110 118 56 114
196 10 148 143 78 192 162 104 185 184 3 151 69 172 133 120
96 121 198 48 72 46 86 117 160 136 8 21 154 140 47 170
125 177 6 132 29 197 175 134 45 60 40 71 155 97 115 180
65 28 149 145 158 116 147 42 50 130 30 183 38 64 85 59
167 51 87 179 82 88 171 99 67 24 34 181 152 73 98 27
135 75 79 156

NN (any position) — objective 52,915 (distance 20,068, costs 32,847), first node 110:
110 2 14 20 74 153 182 52 90 129 127 80 54 192 162 104
76 185 57 89 86 46 72 3 184 173 151 122 117 8 160 69
172 136 133 120 156 43 48 96 198 121 130 191 50 42 147 116
37 146 13 149 28 145 65 158 177 125 26 170 197 36 132 29
6 180 115 155 71 175 134 45 157 60 40 141 97 154 140 47
38 21 176 30 183 189 168 64 59 85 51 73 167 152 34 24
181 179 87 82

Greedy cycle — objective 55,655 (distance 19,634, costs 36,021), first node 174:
174 36 197 6 180 115 170 97 140 47 155 71 175 134 45 157
60 141 40 33 64 21 189 168 69 160 172 136 133 120 8 117
72 46 86 184 3 173 122 151 98 111 27 135 76 185 162 192
80 52 182 20 74 2 93 110 118 14 153 90 129 127 54 104
57 89 156 43 48 96 198 121 63 130 191 183 30 176 38 154
26 125 177 138 158 145 149 147 42 116 37 146 13 28 65 178
103 29 132 1

2-regret (open path) — objective 68,761 (distance 22,801, costs 45,960), first node 94:
94 169 199 70 190 194 77 104 144 185 79 57 83 44 89 156
164 96 198 121 195 50 7 116 37 13 149 28 145 158 138 55
177 53 65 178 103 180 132 1 108 62 175 134 45 157 60 40
64 47 38 21 168 172 8 92 166 72 46 109 3 122 151 98
25 181 73 59 85 82 88 163 99 39 17 74 20 14 2 23
110 118 131 143 148 196 78 54 162 106 127 90 153 182 150 84
66 35 27 135

2-regret (greedy cycle) — objective 68,150 (distance 21,400, costs 46,750), first node 73:
73 167 181 25 98 151 122 128 4 168 183 176 38 47 140 154
161 26 177 170 115 113 155 141 60 157 134 175 62 108 1 132
103 178 65 145 149 13 37 116 147 123 50 195 121 198 58 16
164 43 166 92 120 133 8 117 95 109 3 27 135 76 185 104
77 194 190 70 187 165 105 139 193 114 10 78 54 162 106 150
182 127 90 56 131 110 23 2 14 20 74 17 171 99 163 31
88 82 85 59

Weighted sum (open path) — objective 57,299 (distance 19,735, costs 37,564), first node 105:
105 81 169 70 187 114 10 143 131 118 110 93 2 14 20 74
17 171 39 99 163 88 87 51 85 59 73 167 181 34 25 98
151 122 128 69 172 133 168 183 21 38 154 140 47 64 40 60
157 45 134 175 62 174 1 132 29 197 6 180 115 170 125 55
158 145 28 149 13 146 37 116 42 50 195 121 198 96 48 43
156 32 89 44 83 57 79 185 76 162 159 54 192 106 80 127
129 90 52 182

Weighted sum (greedy cycle) — objective 59,136 (distance 19,558, costs 39,578), first node 159:
159 192 106 80 127 129 90 153 182 150 186 99 39 171 17 74
20 14 2 93 110 118 56 143 148 10 114 193 139 105 169 187

```
70 190 194 77 57 79 86 46 72 166 92 120 133 172 69 168
183 30 191 130 121 195 50 15 158 149 13 28 145 65 178 103
132 197 6 180 115 170 125 26 154 140 47 64 33 40 60 157
175 134 45 85 51 167 179 181 34 25 98 151 173 3 27 135
76 144 104 54
```

8. The weighted-sum criterion: weights

The weighted sum mixes two quantities measured on different scales, so before choosing weights it is worth looking at the data. The objective adds node costs and travel distances; the table below compares their typical magnitude in each instance.

	Mean node cost	Mean distance	cost / distance
Instance			
TSPA	971	1,645	0.59
TSPB	519	1,548	0.34

Node costs and distances are the same order of magnitude (the mean cost is roughly a third to two thirds of the mean distance), and the insertion delta already combines one cost term with one or two distance terms, so the best-change part keeps both halves of the objective balanced by itself. The min–max normalisation in `WEIGHTED_SUM` removes the remaining scale difference between the regret gap and the best change. Because both parts of the objective are already represented inside the best-change term, one would expect the best change to deserve at least as much weight as the regret; the sweep below tests this, including the two endpoints that reproduce the Laboratory-1 methods.

	Instance	TSPA		TSPB	
	Method	Wsum cycle	Wsum path	Wsum cycle	Wsum path
wr	**wb**				
0.000000	**1.000000**	73,793.4	74,347.0	59,649.9	59,475.5
0.300000	**0.700000**	76,670.7	76,657.2	58,498.4	55,513.0
0.500000	**0.500000**	91,064.1	92,603.3	62,870.4	61,457.6
0.700000	**0.300000**	103,838.6	105,280.2	66,621.1	67,038.0
1.000000	**0.000000**	120,378.8	121,360.3	72,484.8	73,649.8

The `wb = 1` row reproduces the extended nearest neighbour (open path) and the greedy cycle (closed cycle) exactly, and `wb = 0` is pure regret. The averages are **not** monotone in the weights. On TSPA the best value is at the pure best-change endpoint (73,793 for the closed-cycle base) and every regret component makes it worse. On TSPB a small regret weight helps: `wr = 0.3, wb = 0.7` gives the best averages of the whole sweep (58,498 closed-cycle and 55,513 open-path, below the 59,650/59,476 of the pure best-change endpoint). The equal weights used for the main results are therefore a neutral default rather than the optimum — they are dominated on both instances, and the data suggests the best change should dominate. This fits the node-cost scale: TSPB's nodes are relatively cheap (cost/distance 0.34), so its objective is dominated by travel and the regret signal adds useful placement information, whereas TSPA's costlier nodes (0.59) make the pure best-change rule already effective.

9. Observations

- TSPA: the lowest average objective is produced by Greedy cycle at 73,793. The pure 2-regret variants are the weakest non-random family — Regret path and Regret cycle average 121,360 and 120,379, about 64% above the leader. Since the node cost is identical at every position it cancels out of the regret, so pure 2-regret is cost- blind and ranks nodes only by the spread of their distance options - the wrong signal when the objective also contains node costs.
- TSPB: the lowest average objective is produced by NN append at 59,397. The pure 2-regret variants are the weakest non-random family — Regret path and Regret cycle average 73,650 and 72,485, about 24% above the leader. Since the node cost is identical at every position it cancels out of the regret, so pure 2-regret is cost- blind and ranks nodes only by the spread of their distance options - the wrong signal when the objective also contains node costs.
- Blending the best change back through the weighted sum recovers most of that loss: on TSPA, Wsum path and Wsum cycle average 92,603 and 91,064, only 23.4–25.5% above Greedy cycle. The criterion now rewards both a small best change and a large regret, which is exactly the trade-off the two Laboratory-1 methods represent at its endpoints.
- Blending the best change back through the weighted sum recovers most of that loss: on TSPB, Wsum path and Wsum cycle average 61,458 and 62,870, only 3.5–5.8% above NN append. The criterion now rewards both a small best change and a large regret, which is exactly the trade-off the two Laboratory-1 methods represent at its endpoints.
- The best single solution on TSPA is 72,328 found by NN (any position).
- The best single solution on TSPB is 52,915 found by NN (any position).
- Cost per construction: append-at-end 0.045 ms, any-position 1.35 ms, greedy cycle 2.58 ms, 2-regret 1.53–3.19 ms and weighted sum 1.59–3.22 ms. The regret rules keep the full cost of the any-position/greedy-cycle search and add a second-best scan, so they are the most expensive without being the most accurate.
- Random stays a baseline: its objective is several times the greedy values on both instances, and both its distance and node-cost terms are far worse, so it fails on both parts of the objective.

Discussion. Keeping the Laboratory-1 constructions as a reference shows that the regret idea does not help in its pure form. Pure 2-regret is the weakest of all non-random methods on both instances: because the node cost cancels out of the regret, the rule is cost-blind and inserts the node with the largest distance spread between its best and second-best option, which favours urgent placement over cheap placement and ends up with worse routes and more expensive node sets. The weighted sum interpolates between pure regret and the Laboratory-1 methods; at equal weights it is clearly better than pure regret but still worse than the greedy methods it generalises. The sweep in Section 8 shows that the equal-weight default is not the optimum: the best change should dominate, and on TSPB a small regret weight (`wr = 0.3`) actually improves on the pure best-change endpoint. Overall the practical conclusion of Laboratory 1 is unchanged — on TSPA the greedy cycle is the most reliable average method, on TSPB append-at-end (and, in the sweep, the `wr = 0.3` weighted sum) trades a longer route for a cheaper node set, and the best single solutions come from the broadest positional search.

10. Reproduction

```
# 1. Python environment (once)
python3 -m venv .venv
.venv/bin/pip install nbconvert ipykernel matplotlib pandas numpy jupyterlab

# 2. Engine
make

# 3. Execute this notebook, export HTML with report.css and print the PDF
.venv/bin/python tools/render_report.py
```

`run.sh` performs all of these steps and can also be used to rebuild everything from scratch; the notebook can be opened for interactive use with `.venv/bin/jupyter lab`. The engine writes `results/summary.csv`, `results/runs.csv`, `results/best_tours.csv` and `results/best_tours_columns.csv`; this notebook writes the figures to `report/figures/` and the exported report to `report/report.html` and `report/report.pdf`.